

Guide to Trajectory Optimization with GODOT and PyGMO

From the optimizer to the configuration:
a backward path to understanding every building block

by Giovanni Guerriero

*Written for those coming from MATLAB who want to understand
how optimization works in Python with GODOT (ESA)*

Logical path of the document

Ch. 1	The endpoint: the call to PyGMO
Ch. 2	cosmos.Problem — the translator between GODOT and PyGMO
Ch. 3	cosmos.Trajectory — the event timeline
Ch. 4	The timeline blocks: control, manoeuvre, match, point
Ch. 5	cosmos.Universe — the physical model
Ch. 6	Free variables, bounds and scales
Ch. 7	The objective function (totalDV)
Ch. 8	The complete flow: putting it all together

Chapter 1 The call to PyGMO

Let's start from the end: the line that launches the optimization. In all the project's scripts, the optimization boils down to this sequence:

```
# 1. Wrap the GODOT problem in a PyGMO object
problem = pg.problem(prob)

# 2. Set the tolerance on the constraints
problem.c_tol = [1e-5] * problem.get_nc()

# 3. Get the initial vector (the guess)
x0 = prob.get_x()

# 4. Create a population and insert the guess
pop = pg.population(problem, 0)
pop.push_back(x0)

# 5. Configure the IPOPT algorithm
ip = pg.ipopt()
ip.set_numeric_option('tol', 1e-3)
algo = pg.algorithm(ip)
algo.set_verbosity(1)

# 6. LAUNCH THE OPTIMIZATION
pop = algo.evolve(pop)
```

These six lines are the heart of the optimization. All the rest of the code serves to prepare the `prob` object that is passed to `pg.problem()`. Let's see piece by piece what each of these lines does.

1.1 — `pg.problem(prob)`: the wrapper

PyGMO knows nothing about spacecraft trajectories. It only knows how to optimize mathematical functions. `pg.problem(prob)` takes the GODOT object `prob` (which contains the objective function, the constraints and the free variables) and "translates" it into the format that PyGMO understands.

MATLAB comparison: In MATLAB, this step doesn't exist because `fmincon` accepts functions directly. In Python, PyGMO needs an object with specific methods (`fitness()`, `get_bounds()`, etc.). GODOT already implements these methods inside `cosmos.Problem`, so `pg.problem()` works automatically.

1.2 — Tolerance on constraints

```
problem.c_tol = [tol_con] * problem.get_nc()
```

`get_nc()` returns the total number of constraints (the match constraints). `c_tol` is a list with the tolerance for each constraint. If you have 72 constraints and `tol_con = 1e-5`, you are saying: "each constraint is satisfied if its value is below 0.00001".

MATLAB comparison: In MATLAB with `fmincon`, the tolerance on the constraints is set via `options.ConstraintTolerance` (a single scalar). PyGMO lets you specify a different tolerance for *each* constraint.

1.3 — The initial guess: `prob.get_x()`

`prob.get_x()` returns a NumPy vector with all the initial values of the free variables. These come from the trajectory configuration (the positions, velocities and ΔV of the initial control points). This is the starting point for the optimizer.

MATLAB comparison: In MATLAB, the initial guess is the vector `x0` that you pass directly to `fmincon(fun, x0, ...)`. Here the concept is identical, but the vector is built automatically by GODOT from the YAML configuration.

1.4 — The population

```
pop = pg.population(problem, 0)    # empty population
pop.push_back(x0)                  # add the guess
```

PyGMO thinks in terms of *populations* of candidate solutions (this comes from the world of genetic algorithms). IPOPT is a gradient-based method (like `fmincon`): it only needs a single individual. So we create an empty population and put only our guess into it.

MATLAB comparison: If you used `ga` in MATLAB, the population would be generated automatically. PyGMO works the same way if you pass a number > 0 to the constructor of `pg.population(problem, 100)`. With IPOPT, a population of a single individual is enough.

1.5 — The IPOPT algorithm

```
ip = pg.ipopt()
ip.set_numeric_option('tol', 1e-3)
algo = pg.algorithm(ip)
algo.set_verbosity(1)
```

IPOPT (Interior Point OPTimizer) is the actual algorithm. It is an interior-point method for nonlinear problems with constraints, similar to the `interior-point` option of `fmincon`. The `tol` tolerance controls when the algorithm stops: when the norm of the gradient of the Lagrangian drops below this value.

Note: The choice of `tol` depends on the context. In the project's code, `1e-3` ("loose") is used when looking for an approximate solution (phase A, feasibility only), and tighter tolerances are used in the final phases. Lowering it too much can slow the optimization down a lot or prevent convergence.

1.6 — `algo.evolve(pop)`: the optimization

This is the line that actually launches the optimization. IPOPT:

1) Evaluates the objective function and the constraints at the current point. 2) Computes the gradients via automatic differentiation. 3) Updates the variables following the descent direction. 4) Repeats until convergence.

After `evolve()`, the population contains the optimized solution. GODOT's `prob` object is updated internally, so you can use `traj.compute()` to recompute the trajectory with the optimized parameters.

Warning: The key question now is: what is the `prob` object that we passed to `pg.problem()`? It's a `cosmos.Problem`, and to understand it we need to take a step back.

Chapter 2 `cosmos.Problem`

`cosmos.Problem` is the GODOT object that contains everything the optimizer needs to know: the objective function, the constraints, the free variables, the bounds and the scales. It's created like this:

```
prob = cosmos.Problem(universe, [traj], config_prob, useGradient=True)
```

Let's look at the four arguments:

Argument	Type	Description
universe	cosmos.Universe	The physical model (Sun, Earth, Moon, gravity, ephemerides)
[traj]	list of Trajectory	The trajectories involved in the problem (here just one)
config_prob	Python dictionary	Defines the objective, free variables, bounds and scales
useGradient	bool	If True, uses automatic differentiation for the gradients

2.1 — The structure of config_prob

The config_prob dictionary is the heart of the problem definition. It can have up to four sections:

```
config_prob = {
    'objective': {          # OPTIONAL: if absent, feasibility only
        'type': 'minimise',
        'point': 'end_pointa',
        'value': 'TotalDV_traj',
        'scale': 1e3
    },
    'parameters': {
        'free': [...],      # REQUIRED: list of free variable names
        'scales': {...},    # OPTIONAL: dictionary name -> scale
        'bounds': {...}     # OPTIONAL: dictionary name -> [min, max]
    }
}
```

Case without an objective (feasibility)

If you omit the objective section, the problem becomes a pure system of equations: IPOPT looks for a point that satisfies the constraints (the match constraints), without minimizing anything. This is the case of "phase A" in the code:

```
config_prob = {'parameters': {'free': [...]]}
# No 'objective' -> feasibility only
```

MATLAB comparison: In MATLAB, this is like calling `fmincon` with a constant objective function (e.g. `@(x) 0`). GODOT handles it automatically: if you don't specify objective, the objective is implicitly zero.

Case with an objective (optimization)

When the objective section is present, IPOPT minimizes the specified function while satisfying the constraints. The fields are:

type: 'minimise' or 'maximise'. point: the timeline point where the objective is evaluated (typically the last one). value: the name of the evaluable registered in the universe (e.g. 'TotalDV_traj'). scale: scale factor to bring the objective to $\sim O(1)$ values.

Note: The value field must match the name under which you registered the objective function in the universe with `universe.evaluables.add('TotalDV_traj', dv_traj)`. If the names don't match, GODOT won't find the function.

2.2 — The two-phase strategy

In the project's code, the transfer trajectory is optimized in two phases:

Phase A (feasibility): `config_prob` is used without an objective. IPOPT finds a point that satisfies the match constraints (assembles the arcs into a continuous trajectory). It doesn't matter whether the ΔV is high or low: only a continuous trajectory is wanted.

Phase B (minimization): `config_prob2` is used with the objective `TotalDV_traj`. Starting from the phase A solution, IPOPT minimizes the total ΔV . In `config_prob2`, `ctr0a_dt` is also added to the free variables, allowing the departure time from the Halo orbit to be optimized.

MATLAB comparison: This two-phase strategy is also common in MATLAB. It corresponds to: (1) solving the system with `fsolve` (constraints only), then (2) optimizing with `fmincon` starting from the `fsolve` solution. In GODOT the two phases use the same framework, only changing `config_prob`.

Chapter 3 `cosmos.Trajectory`

`cosmos.Trajectory` is the object that represents the trajectory. It's created from the universe and a configuration dictionary:

```
traj = cosmos.Trajectory(universe, config_traj)
```

The `config_traj` dictionary has three main sections:

```
config_traj = {
    'settings': {
        'relTol': 1e-12,      # integrator tolerance
        'steps': 10000000    # max integration steps
    },
    'setup': [
        {'name': 'SC',
         'type': 'group',
         'spacecraft': 'SC',
         'input': [
             {'name': 'center', 'type': 'point'},
             {'name': 'dv', 'type': 'scalar', 'unit': 'm/s'}
         ]
        }
    ],
    'timeline': [...]        # the list of events
}
```

3.1 — settings

`relTol` is the relative tolerance of the numerical integrator (the Runge-Kutta integrator internal to GODOT). A value of `1e-12` is very precise, needed for long trajectories where small errors accumulate. `steps` is the maximum number of integration steps: 10 million guarantees that even very long trajectories are integrated without interruption.

3.2 — setup

The `setup` section defines the spacecraft (SC) and its tracked quantities. Here the spacecraft has two state variables:

center (type point): the 3D position of the spacecraft. It has six components: pos_x, pos_y, pos_z, vel_x, vel_y, vel_z.

dv (type scalar): the ΔV accumulated along the trajectory, in m/s. It's updated automatically at every manoeuvre.

3.3 — timeline

The timeline is the heart of the trajectory: a chronologically ordered list of **events**. Each event has a type field that specifies its nature. There are four possible types: control, manoeuvre, match and point. We'll look at them in detail in the next chapter.

3.4 — Important Trajectory methods

Method	What it does
<code>traj.compute(partials=False)</code>	Propagates the trajectory (integrates the equations of motion). With <code>partials=True</code> it also computes the derivatives.
<code>traj.point('ctr0a')</code>	Returns the epoch (Epoch) of the point with that name.
<code>traj.applyParameterChanges()</code>	Returns a dictionary with the changes made by the optimizer, useful for updating the configuration.
<code>traj.getTimelineSolution()</code>	Returns the timeline with the optimized states (for post-optimization analysis).

Chapter 4 The timeline blocks

The timeline is built with four types of blocks. Understanding these blocks is fundamental because they are the elements the optimizer manipulates. The underlying concept is **multiple shooting**: the trajectory is split into short arcs, each defined by an initial control point. The match constraints enforce continuity between the arcs.

4.1 — Control Point (type: 'control')

A control point defines the state of the spacecraft (position + velocity) at a specific epoch. It's a "node" of the trajectory: the integrator starts here and propagates forward.

```
{
  'type': 'control',
  'epoch': '8872.03894 TDB',      # epoch in MJD
  'name': 'ctr0a',                # unique name
  'state': [
    {'name': 'SC_center',
     'point': 'Earth',            # coords relative to Earth
     'coi': 'Earth',             # center of integration
     'axes': 'SEROT',            # Sun-Earth rotating frame
     'project': False,
     'dynamics': 'EMS_gravity',  # dynamics model
     'value': {
```

```

        'pos_x': '1665040.73 km',
        'pos_y': '-8535.85 km',
        'pos_z': '577412.76 km',
        'vel_x': '0.01392 km/s',
        'vel_y': '-0.32436 km/s',
        'vel_z': '0.00428 km/s'
    }},
    {'name': 'SC_dv', 'value': '0 m/s'}
]
}

```

The important fields of the state are: **point** and **coi** (center of integration): define with respect to which body the coordinates are expressed and from which body gravity "starts" in the integration. **axes**: the reference frame (here SEROT = Sun-Earth rotating). **dynamics**: the gravity model to use (here EMS gravity = Sun + Earth + Moon). **value**: the six components of the initial state with their units.

Note: The names of the control points (ctr0a, ctr1a, ...) become the prefixes of the free variables. For example, the x position of ctr1a will be called ctr1a_SC_center_pos_x. This naming system is fundamental for connecting the timeline blocks to the optimizer's variables.

4.2 — Manoeuvre (type: 'manoeuvre')

An impulsive manoeuvre: an instantaneous ΔV applied to the spacecraft. It changes the velocity without changing the position.

```

{
  'type': 'manoeuvre',
  'name': 'man_injection',
  'model': 'impulsive',
  'input': 'SC',
  'thruster': 'main',
  'config': {
    'point': {
      'reference': 'ctr0a',    # applied to control point ctr0a
      'dt': '0 s'            # instantaneously (dt = 0)
    },
    'direction': {
      'axes': 'SEROT',
      'dv_x': '-0.0107 km/s',
      'dv_y': '0.0036 km/s',
      'dv_z': '-0.0059 km/s'
    }
  }
}

```

The reference field attaches the manoeuvre to a specific control point. dt is the delay relative to the reference (0 s = instantaneous). The three components dv_x, dv_y, dv_z are the variables that the optimizer can modify if they are included in the free variables.

4.3 — Match (type: 'match')

The match constraint is the heart of multiple shooting. It enforces that the forward propagation from a control point and the backward propagation from the next one meet at the same point.

```

{
  'type': 'match',
  'name': 'match0a',

```

```

'input': 'SC',
'left': {
  'reference': 'ctr0a',
  'dt': '1740263.33 s'    # propagate forward by dt
},
'right': {
  'reference': 'ctr1a',
  'dt': '1740263.33 s'    # propagate backward by dt
},
'body': 'Earth',
'vars': 'cart'
}

```

Logic: GODOT propagates ctr0a forward by dt seconds and ctr1a backward by dt seconds. The constraint requires the two resulting states to be equal (6 equations: 3 for position + 3 for velocity). Typically dt = half the interval between the two control points.

MATLAB comparison: In MATLAB, the match constraints would correspond to the `ceq` function passed to `fmincon(..., @nonlcon)`. In GODOT you don't have to write this function: it's generated automatically from the timeline. Each match generates 6 equality constraints (`ceq` in MATLAB, equality constraints in PyGMO).

4.4 — Point (type: 'point')

A terminal point: defines an evaluation epoch (for example to compute the objective function). It doesn't add constraints or manoeuvres.

```

{
  'type': 'point',
  'input': 'SC',
  'name': 'end_pointa',
  'point': {'reference': 'ctr6a', 'dt': '0 s'}
}

```

In the example, `end_pointa` coincides with the last control point. It is used as the objective evaluation point: `'point': 'end_pointa'` in the objective dictionary.

4.5 — Visual scheme of the timeline

```

ctr0a -man_inj-> [propagate] <-match0a-> [propagate] ctr1a
                        ^meeting point

ctr1a -----> [propagate] <-match1a-> [propagate] ctr2a

ctr2a -----> [propagate] <-match2a-> [propagate] ctr3a -man_dsm->

ctr3a -man_dsm-> [propagate] <-match3a-> [propagate] ctr4a

... -----> [propagate] <-match5a-> [propagate] ctr6a = end_pointa

```

Timeline scheme for the transfer trajectory with $n_{pt_traj} = 3$.

Chapter 5 cosmos.Universe

The universe is the physical model of the solar system. It contains the celestial bodies, the ephemerides, the reference frames and the gravitational constants. It's loaded from a YAML file:

```
config_uni = util.load_yaml('Universe/universe.yml')
universe = cosmos.Universe(config_uni)
```

5.1 — Structure of universe.yml

The file is organized into sections. Let's look at them one by one, starting from the project's file.

spacetime and ephemeris

`spacetime`: `system`: BCRS indicates the fundamental reference system (Barycentric Celestial Reference System, standard for solar system dynamics). The `ephemeris` section loads the ephemeris files: `de432s.bsp` (planet positions) and `gm_de431.tpc` (gravitational constants).

frames

Defines the reference frames. In the project the following are used: **SEROT** (Sun-Earth ROTating): a rotating frame centered on the Sun where Earth is fixed on the X axis. It's the natural frame for Halo orbits. **EMROT** (Earth-Moon ROTating): analogous but for the Earth-Moon system.

constants, bodies, gravity, dynamics

`constants` defines the gravitational parameters (GM) of the bodies. `bodies` lists the celestial bodies. `gravity` defines the gravity model (here `gravEMS` = Sun + Earth + Moon). `dynamics` associates the gravity model with a name (`EMS_gravity`) used in the timeline's control points.

spacecraft

Defines the spacecraft: mass (1 kg, fictitious since consumption is not modeled), thruster (fictitious thrust and `Isp` for impulsive manoeuvres). This section is necessary to define manoeuvres in the timeline.

5.2 — universe.evaluables

The universe has a registry of *evaluables*: functions that can be evaluated at an epoch and that are automatically differentiable. They are added with:

```
dv_traj = totalDV_traj(universe)
universe.evaluables.add('TotalDV_traj', dv_traj)
```

Once registered, the function can be called with:

```
val = universe.evaluables.get('TotalDV_traj').eval(epoch)
```

This mechanism is crucial for the optimization objective: the `value` field in `config_prob` points to the name registered here.

Warning: If you recreate the universe (as is done in the code between the two optimizations), you must re-register the evaluables. The new universe is "clean" and doesn't know the functions of the old one.

5.3 — universe.frames

The universe lets you compute positions and distances between celestial bodies:

```
# SC state relative to Earth in the SEROT frame
x = universe.frames.vector6('Earth', 'SC_center', 'SEROT', epoch)
```

```
# -> [pos_x, pos_y, pos_z, vel_x, vel_y, vel_z] in km and km/s

# Sun-Earth distance
d = universe.frames.distance('Sun', 'Earth', epoch)
# -> distance in km
```

5.4 — NASA/JPL ephemeris files: catalogue and types

Planetary ephemerides are numerical tables providing the position and velocity of solar system bodies as a function of time. They are produced by NASA's JPL (Jet Propulsion Laboratory) through numerical integration of the equations of motion, calibrated against telescopic observations and spacecraft data. They are distributed in the SPICE format, developed by the NAIF (Navigation and Ancillary Information Facility) at JPL.

The Development Ephemerides (DE)

The "DE" (Development Ephemeris) series is JPL's main family of planetary ephemerides. Each new version incorporates more recent observational data and improved dynamical models. The files contain the barycentric positions (relative to the solar system barycenter) of the Sun, the eight planets and Pluto, plus the geocentric coordinates of the Moon. The project's file uses `de432s.bsp`. Here is a catalogue of the most relevant versions:

Version	Year	Time coverage	Size	Notes
DE405	1997	1600 - 2200	~56 MB	Historical standard, used in the Astronomical Almanac 2003-2014
DE421	2008	1900 - 2050	~17 MB	Compact, very popular for generic applications
DE430	2013	1550 - 2650	~120 MB	High accuracy, includes lunar liquid-core effects. Almanac standard since 2015
DE431	2013	13201 BC - 17191 AD	~3.4 GB	Long-term version of DE430, less accurate near the current epoch
DE432	2014	1550 - 2650	~120 MB	Update of DE430 for the New Horizons mission (Pluto). Used in the project
DE436	2016	1550 - 2650	~120 MB	Based on DE430 with improved Jupiter orbit for the Juno mission
DE440	2020	1550 - 2650	~114 MB	Most recent, 7 more years of data compared to DE430. Improved orbits of Jupiter, Saturn and Pluto. Current standard
DE441	2020	13200 BC - 17191 AD	~3.2 GB	Long-term version of DE440, less accurate near the current epoch

Note: Rule of thumb: for missions between 1550 and 2650, use DE440 (or DE432/DE430). For dates outside this range, use DE441 (or DE431). The "short" versions (`de440s.bsp`, `de432s.bsp`) cover roughly 1950-2050 and are only ~10 MB: they're ideal during development. For the Comet Interceptor project (2029 flyby), DE432s is sufficient.

The differences between successive versions are generally very small (on the order of a few km for the outer planets, fractions of a km for the inner ones). The choice of version becomes important only for very

high-precision missions (interplanetary navigation, radio astronomy) or when you need to use small-body SPK files generated with a specific DE.

The SPICE kernel types

Ephemerides are distributed in the SPICE format, which provides for different file types (called "kernels"). The project uses two, but it's useful to know the whole ecosystem since you might need it in the future. Each kernel type has a conventional extension:

Type	Extension	Format	Content	Used in the project?
SPK	.bsp	binary	Position and velocity of celestial bodies (planets, satellites, comets, asteroids, spacecraft) as a function of time. This is the actual ephemeris file.	Yes — de432s.bsp
PCK	.tpc / .bpc	text / binary	Physical constants of the bodies: gravitational parameters (GM), radii, rotation parameters. The .tpc file is readable with a text editor.	Yes — gm_de431.tpc
FK	.tf	text	Reference frame definitions (relations between frames, orientations). Useful for custom frames.	No (frames are defined in universe.yml)
LSK	.tls	text	Leap Second Kernel: table of leap seconds to convert between UTC and TDB/ET. Needed if working in UTC.	No (GODOT handles this internally)
CK	.bc	binary	Camera/pointing Kernel: orientation of spacecraft and instruments over time. Used for attitude reconstruction.	No
DSK	.bds	binary	Digital Shape Kernel: 3D shape models of bodies (asteroids, comets, moons). Used for proximity analysis.	No
SCLK	.tsc	text	Spacecraft Clock: conversion between the spacecraft's onboard clock and ephemeris time.	No
MK	.tm	text	Meta-Kernel: a file that lists all the other kernels to load together. Convenient for organizing many files.	No

In GODOT, kernels are loaded in the `ephemeris` section of `universe.yml`. The project loads two kernels: an SPK for the planet positions and a text-based PCK for the gravitational constants. They can be loaded in separate blocks (as in the project's file) or together if they are of the same type.

Note: SPICE files can be downloaded from JPL's NAIF server: https://naif.jpl.nasa.gov/pub/naif/generic_kernels/. Planetary ephemerides (SPK) are found in the `spk/planets/` subfolder, satellite ones in `spk/satellites/`, and physical constants in `pck/`.

How the project loads the kernels in `universe.yml`

```
ephemeris:
  - name: de432          # internal reference name
    files:
      - /path/de432s.bsp # SPK file (planet positions)
    cache: True
  - name: gm431
    files:
      - /path/gm_de431.tpc # text PCK file (GM constants)
```

Each block has a name that is then referenced in the constants section (for PCK) or frames section (for SPK). The `cache: True` field loads the kernel into memory to speed up repeated access.

5.5 — Adding planets and bodies to the model

The project's `universe.yml` file only includes Sun, Earth and Moon because they are enough for the EMS (Earth-Moon-Sun) gravity model used for trajectories near L2. But many missions require adding other planets (for example Jupiter for a gravity assist). The procedure is simple and requires modifying four sections of the file.

Step 1: Make sure the ephemerides contain the body

The DE files (`de430.bsp`, `de432s.bsp`, `de440.bsp`, etc.) already contain **all eight planets** plus Pluto and the Moon. So to add Jupiter, Mars or any other planet, there's no need to download a new SPK file: you just need to add the body in the following sections.

If instead you want to add a **natural satellite** (for example Europa, Enceladus, Titan), you need to download the dedicated SPK file from the NAIF server. For example, for Saturn's satellites you would use `sat441.bsp`, adding it to the ephemeris section:

```
ephemeris:
  - name: de440
    files:
      - /path/de440s.bsp
    cache: True
  - name: sat_eph          # NEW: satellite ephemerides
    files:
      - /path/sat441.bsp
    cache: True
  - name: gm431
    files:
      - /path/gm_de431.tpc
```

Step 2: Add the body to the bodies section

The bodies section lists the celestial bodies GODOT needs to know about. Each body has a name (used in the rest of the file) and a point (the name in the SPICE ephemerides). The standard SPICE names are: Sun, Mercury, Venus, Earth, Moon, Mars, Jupiter, Saturn, Uranus, Neptune, Pluto.

```
bodies:
  - name: Earth
    point: Earth
  - name: Sun
    point: Sun
  - name: Moon
    point: Moon
  - name: Jupiter          # NEW
```

```

point: Jupiter
- name: Saturn          # NEW
point: Saturn

```

Step 3: Add the GM to the constants section

Every body that exerts gravity needs its gravitational parameter GM. If you load the `gm_de431.tpc` file, the GM values of all the planets are already available via the source: `gm431` line. However, if you want to override a value (as the project does for Earth and the Sun), you specify it explicitly in the data section:

```

constants:
  ephemeris:
    - source: gm431    # load GM values from gm_de431.tpc
  data:
    EarthGM: 398702.4418    # override (km^3/s^2)
    SunGM: 132758540018
    MoonGM: 4902.7779
    JupiterGM: 126686534    # NEW (from the tpc file or literature)
    SaturnGM: 37931187     # NEW

```

Note: The GM values in the `gm_de431.tpc` file are expressed in km^3/s^2 . You can read them by opening the file with a text editor: it's a text PCK file with variables in the format `BODY599_GM = (126686534.911)` (599 is Jupiter's NAIF code). If you specify a value in the data section, it overrides the one from the file.

Step 4: Update gravity and dynamics

The gravity section defines *which bodies* exert gravitational force in the model. The dynamics section associates the gravity model with a name used in the timeline. To add Jupiter and Saturn as perturbbers:

```

gravity:
  - name: gravFull          # model name
  bodies:
    - Sun
    - Earth
    - Moon
    - Jupiter               # NEW
    - Saturn               # NEW

dynamics:
  - name: full_gravity      # name used in the timeline
    type: SystemGravity
    config:
      model: gravFull

```

After this change, in the timeline's control points you can use `dynamics: full_gravity` instead of `dynamics: EMS_gravity` to include the perturbations from Jupiter and Saturn in the propagation.

Warning: You can define **multiple gravity models** in the same file and use the appropriate one depending on the mission phase. For example: `EMS_gravity` for the Halo phase near L2 (where Jupiter is negligible) and `full_gravity` for the transfer to the comet. Each control point in the timeline can specify a different dynamics model.

Complete example: universe.yml with Jupiter and Saturn

```

version: '3.0'
spacetime:

```

```
system: BCRS

ephemeris:
- name: de440
  files:
    - /path/de440s.bsp
  cache: True
- name: gm431
  files:
    - /path/gm_de431.tpc

frames:
- name: ephem1
  type: Ephem
  config:
    source: de440
- name: SEROT
  type: AxesLocalOrbital
  config:
    center: Sun
    target: Earth
    axes: Pos

constants:
  ephemeris:
    - source: gm431
  data:
    EarthGM: 398702.4418
    SunGM: 132758540018
    MoonGM: 4902.7779

bodies:
- name: Earth
  point: Earth
- name: Sun
  point: Sun
- name: Moon
  point: Moon
- name: Jupiter
  point: Jupiter
- name: Saturn
  point: Saturn

gravity:
- name: gravEMS
  bodies: [Sun, Earth, Moon]
- name: gravFull
  bodies: [Sun, Earth, Moon, Jupiter, Saturn]

dynamics:
- name: EMS_gravity
  type: SystemGravity
  config:
    model: gravEMS
- name: full_gravity
  type: SystemGravity
  config:
    model: gravFull
```

```
spacecraft:
  - name: SC
    mass: 1 kg
    thrusters:
      - name: main
        thrust: 1 N
        isp: 1 s
```

Complete universe.yml example with two gravity models: EMS (near L2) and full (transfer).

MATLAB comparison: In MATLAB, adding a planet to the gravity model corresponds to adding a term in the equations of motion function (the "right-hand side" of the ODE). In GODOT you don't touch the code: everything is done by modifying the YAML file. GODOT takes care of automatically adding the gravitational terms to the integrator.

5.6 — How GODOT finds bodies in the kernels (NAIF ID)

This section answers a natural question: if I add a satellite like Enceladus to the bodies section and load sat441.bsp in the ephemerides, how does GODOT know it has to look for Enceladus specifically in that file?

The answer is: **GODOT does not associate bodies with files by file name**. The link is made through **NAIF codes**, a standard numerical identification system defined by JPL for every body in the solar system.

The mechanism in three steps

1. Loading the kernels: When GODOT loads an SPK file (for example sat441.bsp), it internally registers all the *segments* contained in the file. Each segment is identified by a numerical NAIF code, not by the file name. For example, sat441.bsp contains segments for codes 601 (Mimas), 602 (Enceladus), 603 (Tethys), etc.

2. Resolving the name: When you write point: Enceladus in the bodies section, GODOT (via SPICE) translates the name "Enceladus" into the corresponding NAIF code: 602.

3. Searching the loaded kernels: GODOT looks through *all* the loaded SPK kernels for the one that contains data for code 602. It doesn't matter what the file is called: it could be sat441.bsp, my_file.bsp or any other name. What matters is that the file contains the segment with that NAIF code.

Note: In practice: the point field in the bodies section must use the body's **standard SPICE name** (for example "Enceladus", the English SPICE name). SPICE has an internal name-to-code table. If you use an unrecognized name, GODOT won't know which code to look for. The name field, on the other hand, is free: you can call it whatever you like ("Enceladus", "enc", "target") because it's just an alias used in the rest of the YAML file.

```
bodies:
  - name: Enceladus      # free name, used in the YAML
    point: Enceladus    # SPICE name -> translated to code 602
                        # GODOT looks for code 602 in all SPK kernels
```

Diagram of the mechanism

universe.yml	GODOT/SPICE internally
=====	=====

```

ephemeris:
  files: de440s.bsp    ---->
  files: sat441.bsp   ---->

Loading kernels:
Registers segments: 1,2,3,...399,301,10
Registers segments: 601,602,603,...699

bodies:
  point: Enceladus    ---->

Resolution:
Name 'Enceladus' -> NAIF code 602
Looks for 602 in the loaded kernels
-> found in sat441.bsp -> OK!

```

Table of the most common NAIF codes

Here are the NAIF codes of the bodies you might want to add to the model. The barycenters of the planetary systems have codes 1-9, while the planets themselves have codes x99. Satellites have codes xNN where x is the planet and NN the satellite number.

Body	SPICE name	NAIF code	SPK file
Sun	Sun	10	de4xx.bsp
Mercury	Mercury	199	de4xx.bsp
Venus	Venus	299	de4xx.bsp
Earth	Earth	399	de4xx.bsp
Moon	Moon	301	de4xx.bsp
Mars	Mars	499	de4xx.bsp
Jupiter	Jupiter	599	de4xx.bsp
Saturn	Saturn	699	de4xx.bsp
Uranus	Uranus	799	de4xx.bsp
Neptune	Neptune	899	de4xx.bsp
Pluto	Pluto	999	de4xx.bsp
Io	Io	501	jup365.bsp
Europa	Europa	502	jup365.bsp
Ganymede	Ganymede	503	jup365.bsp
Callisto	Callisto	504	jup365.bsp
Mimas	Mimas	601	sat441.bsp
Enceladus	Enceladus	602	sat441.bsp
Tethys	Tethys	603	sat441.bsp
Dione	Dione	604	sat441.bsp
Rhea	Rhea	605	sat441.bsp
Titan	Titan	606	sat441.bsp

Satellite SPK files can be downloaded from https://naif.jpl.nasa.gov/pub/naif/generic_kernels/spk/satellites/. The file name indicates the planetary system and version: jup365.bsp for Jupiter (version 365), sat441.bsp for Saturn (version 441), and so on.

What happens if the kernel is missing?

If you write `point: Enceladus` but haven't loaded an SPK file containing code 602, GODOT will raise a runtime error when it tries to compute Enceladus's position. The error typically indicates that no SPK segment was found for the requested code within the specified time interval.

Warning: Watch out for **time coverage**: even if the kernel contains the right body, the data might not cover your mission's dates. For example, `sat441.bsp` covers a specific interval. If your trajectory falls outside this interval, you'll get an error. Always check that the kernel's coverage includes the mission's dates.

Concrete example: adding Enceladus to the model

```
# In universe.yml:

ephemeris:
  - name: de440
    files: [/path/de440s.bsp]
    cache: True
  - name: sat_eph
    files: [/path/sat441.bsp]    # contains codes 601-699
    cache: True
  - name: gm431
    files: [/path/gm_de431.tpc]

bodies:
  - name: Sun
    point: Sun                # code 10 -> found in de440s.bsp
  - name: Saturn
    point: Saturn            # code 699 -> found in de440s.bsp
  - name: Enceladus
    point: Enceladus         # code 602 -> found in sat441.bsp
  - name: Titan
    point: Titan             # code 606 -> found in sat441.bsp

gravity:
  - name: gravSatSystem
    bodies: [Sun, Saturn, Enceladus, Titan]

dynamics:
  - name: saturn_system_gravity
    type: SystemGravity
    config:
      model: gravSatSystem
```

Example for a moon tour around Enceladus: GODOT resolves each SPICE name into a NAIF code and automatically looks for the matching segment in the loaded kernels.

Chapter 6 Free variables, bounds and scales

This chapter is fundamental: it defines *what* the optimizer can change and *by how much* it can change it.

6.1 — Free variables (free)

The free list contains the **names** of the variables that the optimizer can change. Each name follows GODOT's convention:

Variable name	Meaning
ctr1a_SC_center_pos_x	x position of control point ctr1a
ctr1a_SC_center_vel_y	y velocity of control point ctr1a
ctr1a_dt	Time offset of ctr1a relative to the nominal epoch
ctr1a_SC_dv	ΔV accumulated at ctr1a
man_injection_dv_x	x component of the injection manoeuvre's ΔV
man_dsm_dv_z	z component of the DSM's ΔV
match0_right_dt	Propagation time of the right side of match0

The rule is: **if a variable is not in the free list, it's fixed**. For example, if you don't put `ctr0a_dt` in the list, the timing of the starting point will remain whatever was in the initial guess.

MATLAB comparison: In MATLAB, the free variables are simply the elements of the vector `x` passed to `fmincon`. In GODOT, the variables are identified by *name* (strings), and GODOT takes care of mapping them into a numerical vector internally.

Note: A strategic choice in the code: for the last control point (the comet intercept), only the **velocities** are left free, not the positions. This is because the intercept position is fixed (the comet is there), but the velocities need to be able to vary to satisfy the match constraints.

6.2 — Scales

Scales are used to normalize the variables for the optimizer. If one variable has a typical value of 1,500,000 km and another has 0.01 km/s, the optimizer will struggle to treat them fairly. Scales bring everything to $\sim O(1)$ values.

```
SS = {
    'ctr1_SC_center_pos_x': '2000000 km',    # pos_x is about ~2M km
    'ctr1_SC_center_vel_x': '0.2 km/s',      # vel_x is about ~0.2 km/s
    'ctr1_dt': '1 day',                      # dt is about ~1 day
    'man0_dv_x': '1 mm/s',                   # dv is about ~1 mm/s
}
config_prob['parameters']['scales'] = SS
```

MATLAB comparison: In MATLAB with `fmincon`, scales are handled with `options.TypicalX` (which does essentially the same thing). `ga` also benefits from normalized variables.

6.3 — Bounds

Bounds limit the range of each variable. In GODOT they are specified as `[min, max]` pairs with units:

```
BB = {
    'ctr1_SC_center_pos_x': ['1400000 km', '1600000 km'],
    'ctr1_dt': ['-1 day', '1 day'],
    'man0_dv_x': ['-1 m/s', '1 m/s'],
    'ctr1_SC_dv': ['0 m/s', '1000 m/s'],    # always >= 0
}
config_prob['parameters']['bounds'] = BB
```

Bounds are centered on the initial guess value and have a half-width defined by `delta_vect`. For example, if the guess for `pos_x` is 1,500,000 km and `delta = 100,000 km`, the bounds will be [1,400,000, 1,600,000] km.

MATLAB comparison: Bounds correspond directly to the `lb` and `ub` parameters of `fmincon(fun, x0, A, b, Aeq, beq, lb, ub)`. The concept is identical.

Warning: If you don't specify scales and bounds (as in the `config_trajectory()` function), GODOT uses default values. This can work but makes the optimization less controlled and potentially slower.

Chapter 7 The objective function (totalDV)

The objective function is what the optimizer minimizes. In the project there are two different objective functions, implemented as Python classes in `totalDV.py`.

7.1 — How to write an objective function for GODOT

A GODOT objective function must inherit from `common.ScalarTimeEvaluable` and implement the `eval(self, e)` method that returns a scalar value. The trick is that it must use the `autodif` functions (`ad.sqrt`, not `np.sqrt`) to be automatically differentiable.

```
import godot.model.common as common
from godot.core import autodif as ad

class totalDV_traj(common.ScalarTimeEvaluable):

    def __init__(self, universe):
        super().__init__()
        self.universe = universe

    def eval(self, e):
        dv_tot = 0.0

        # Injection DV components
        dvx = self.universe.evaluables.get('man_injection_dv_x').eval(e)
        dvy = self.universe.evaluables.get('man_injection_dv_y').eval(e)
        dvz = self.universe.evaluables.get('man_injection_dv_z').eval(e)
        dv_tot += ad.sqrt(dvx*dvx + dvy*dvy + dvz*dvz)

        # DSM DV components
        dvx = self.universe.evaluables.get('man_dsm_dv_x').eval(e)
        dvy = self.universe.evaluables.get('man_dsm_dv_y').eval(e)
        dvz = self.universe.evaluables.get('man_dsm_dv_z').eval(e)
        dv_tot += ad.sqrt(dvx*dvx + dvy*dvy + dvz*dvz)

        return dv_tot
```

Key points

- 1. Inheriting from `ScalarTimeEvaluable`:** mandatory to register the function in the universe. The `eval(self, e)` method receives an epoch `e` as an argument (even though in this case it isn't used directly).
- 2. Using `ad.sqrt` instead of `np.sqrt`:** fundamental! The operations must be differentiable to compute the gradients. If you use `np.sqrt`, GODOT won't be able to compute the derivatives and gradient-based optimization won't work.

3. Retrieving values via `evaluables.get()`: names like 'man_injection_dv_x' are automatically generated by GODOT when you build the timeline. The name follows the pattern: `manoeuvre_name_component`.

MATLAB comparison: In MATLAB, the objective function is simply a function handle: `@(x) my_function(x)`. In GODOT, the function has access to the whole universe via `self.universe`, which makes it more powerful but requires it to comply with the `ScalarTimeEvaluable` interface. Automatic differentiation replaces the gradient computation that in MATLAB you'd do with `GradObj` in `fmincon`'s options.

7.2 — Registering it in the universe

After creating the instance, it must be registered:

```
dv_traj = totalDV_traj(universe)
universe.evaluables.add('TotalDV_traj', dv_traj)
```

The name 'TotalDV_traj' must then appear in the value field of the objective in `config_prob`. This link is fundamental: if the names don't match, the optimization won't work.

Chapter 8 The complete flow

Now that we've seen every piece, let's summarize the complete logical flow. This is the order in which you need to prepare things to launch an optimization with GODOT and PyGMO.

Step 1: Create `universe.yml`

Define the celestial bodies, the ephemerides, the reference frames, the gravitational constants and the spacecraft.

Step 2: Load the Universe

```
config_uni = util.load_yaml('universe.yml')
universe = cosmos.Universe(config_uni)
```

Step 3: Write the objective function

Create a class that inherits from `ScalarTimeEvaluable` and implements `eval()`. Use `ad.sqrt` and the `autodif` functions. Register it with `universe.evaluables.add()`.

Step 4: Build the timeline

Create the list of events (control, manoeuvre, match, point) describing the trajectory. Each control point has an initial state (the guess).

Step 5: Define `config_prob`

Specify: objective (what to minimize), free (free variables), scales (normalization), bounds (limits).

Step 6: Create Trajectory and Problem

```
traj = cosmos.Trajectory(universe, config_traj)
prob = cosmos.Problem(universe, [traj], config_prob, useGradient=True)
```

Step 7: Launch PyGMO

```

problem = pg.problem(prob)
pop = pg.population(problem, 0)
pop.push_back(prob.get_x())
pop = pg.algorithm(pg.ipopt()).evolve(pop)

```

Step 8: Extract the solution

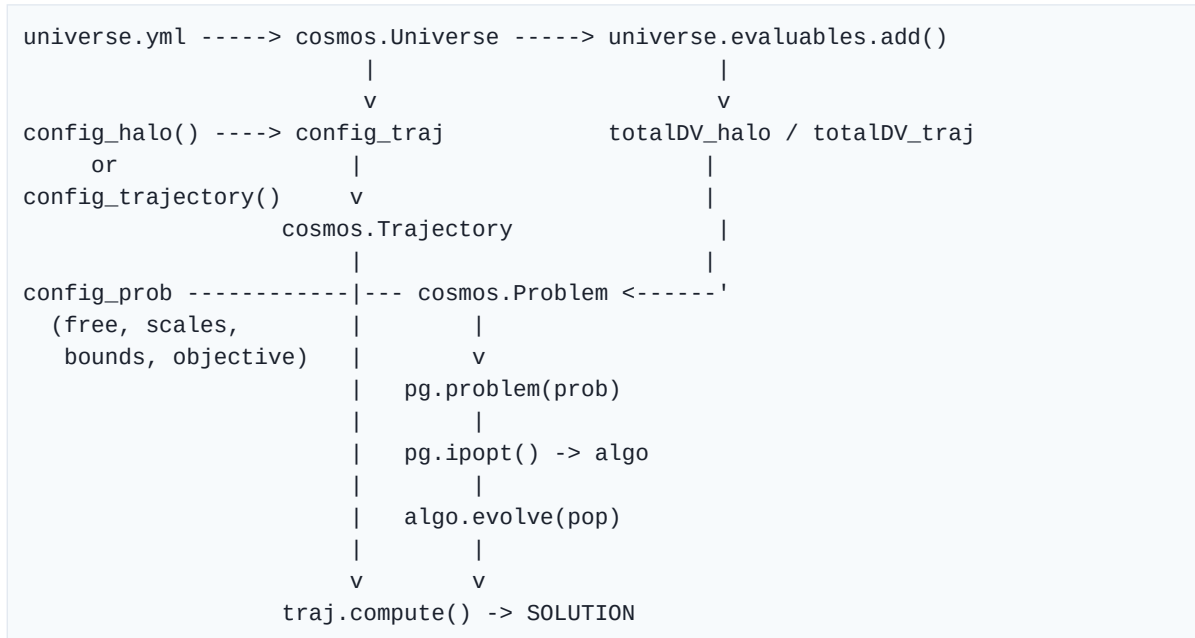
```

traj.compute(partials=False)
traj_up = traj.applyParameterChanges()
conf_update = util.deep_update(config_traj, traj_up)
util.save_yaml(conf_update, 'solution.yaml')

```

8.1 — Dependency map

The following diagram shows how the various objects depend on each other, from the configuration files to the final PyGMO call:



8.2 — Summary table: MATLAB vs GODOT/PyGMO

Concept	MATLAB (fmincon)	GODOT + PyGMO
Objective function	@(x) fun(x)	ScalarTimeEvaluable class + registration in universe
Equality constraints	ceq in nonlcon	Match constraints (generated automatically from the timeline)
Free variables	Elements of vector x	List of names (strings) in config_prob['parameters']['free']
Initial guess	x0	prob.get_x() (built from the timeline)
Bounds	lb, ub	Dictionary with names and values with units

Scales	TypicalX	scales dictionary in config_prob
Algorithm	interior-point	pg.ipopt()
Gradients	GradObj or finite diff.	Automatic differentiation (autodif, with useGradient=True)
Constraint tol.	ConstraintTolerance	problem.c_tol (one per constraint)
Convergence tol.	OptimalityTolerance	ip.set_numeric_option('tol', ...)

Final notes

This guide covers the complete flow from the optimizer to the configuration. The most important points to remember are:

1. Everything revolves around the `config_prob` dictionary: this is where you define what to optimize (objective), what can vary (free), and within what limits (bounds/scales).
2. The timeline is the bridge between physics and optimization: the control points define the states, the matches enforce continuity, the manoeuvres are the optimizer's levers.
3. The universe must contain everything: ephemerides, gravity, spacecraft and objective functions registered as evaluables.
4. Always use `ad.sqrt` (and the `autodif` functions) in the objective function, never `np.sqrt`, otherwise the gradients won't be computed.
5. The two-phase strategy (feasibility first, then minimization) is a good practice that makes the optimization more robust.